Case Study 6: Password Policy Validation Engine

Activity sheet case study #29 | Concepts: Strings (indexing, methods, membership), lists, loops, conditional statements

1. Problem Statement

Evaluate a list of sample password strings against a security policy covering minimum and maximum length, character categories, repeated characters and prohibited substrings. For every password the program must state whether it is valid and explain each failed condition.

2. Objectives

- Traverse strings character by character and classify characters.
- Use string slicing, indexing and the in operator for pattern checks.
- Collect all failed conditions in a list instead of stopping at the first.
- Report results and a summary count.

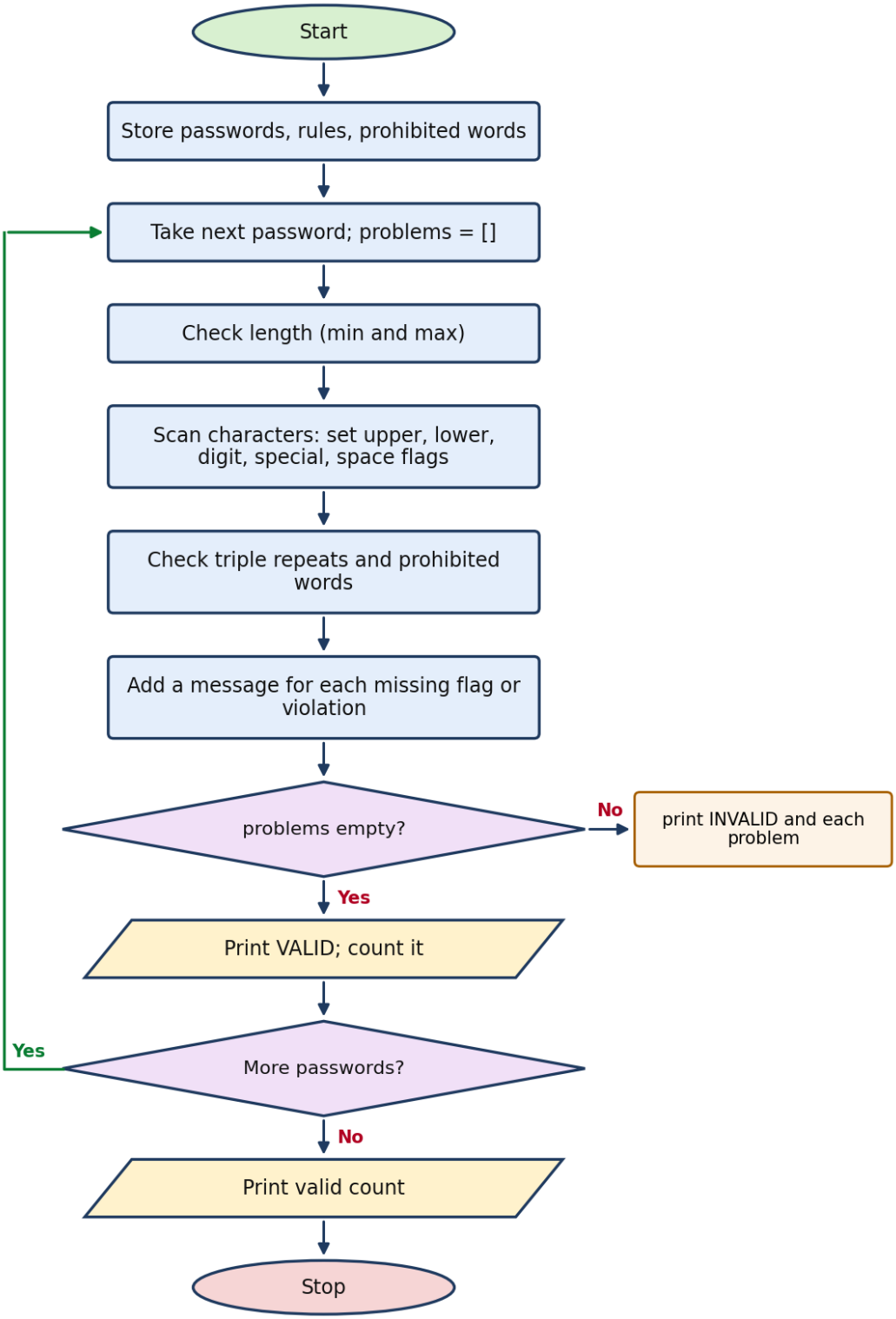
3. Assumptions

- Length must be between 8 and 20 characters.
- At least one uppercase letter, one lowercase letter, one digit and one special character from !@#$%^&*.
- No character may appear three or more times in a row; spaces are not allowed.
- Prohibited substrings (case-insensitive): password, 1234, admin, qwerty.
- Characters outside these groups (for example _) are tolerated but do not count as special.

4. Algorithm

1. Store the passwords, special-character string, prohibited words and length limits.
2. For each password create an empty problems list.
3. Check length against the minimum and maximum.
4. Scan the characters and set flags for uppercase, lowercase, digit, special and space.
5. Scan positions i, i+1, i+2 to detect three equal characters in a row.
6. Convert to lowercase and test every prohibited word with the in operator.
7. Add a message to problems for every failed rule.
8. If problems is empty print VALID, otherwise INVALID followed by each problem.
9. Print how many passwords were valid.

5. Flowchart



6. Pseudocode

```
START
  SET MIN=8, MAX=20, SPECIALS="!@#$%^&*", PROHIBITED=[password,1234,admin,qwerty]
  valid_count <- 0
  FOR each pwd in passwords
      problems <- empty list
      IF LENGTH(pwd) < MIN THEN add "too short"
      IF LENGTH(pwd) > MAX THEN add "too long"
```

```
      FOR each character ch in pwd
          classify ch as upper / lower / digit / special / space
      IF no upper THEN add message  (same for lower, digit, special)
      IF space found THEN add message
      FOR i = 0 TO LENGTH(pwd)-3
          IF pwd[i] = pwd[i+1] = pwd[i+2] THEN add "repeated" ; BREAK
      FOR each word in PROHIBITED
          IF word is inside LOWERCASE(pwd) THEN add message
      IF problems is empty THEN PRINT VALID ; valid_count <- valid_count + 1
      ELSE PRINT INVALID and every problem
  END FOR
  PRINT valid_count
STOP
```

7. Python Source Code

```
# Case Study 29: Password Policy Validation Engine
# ---- INPUT DATA ----
passwords = ["Str0ng@Pass", "weak", "PASSWORD123!", "Aaaa1111!!",
             "Hello World1!", "Admin@2024x", "Tr1cky#Pw", "Qwerty!9Zx"]
# ---- END DATA ----

SPECIALS = "!@#$%^&*"
PROHIBITED = ["password", "1234", "admin", "qwerty"]
MIN_LENGTH = 8
MAX_LENGTH = 20

valid_count = 0
for pwd in passwords:
    problems = []

    if len(pwd) < MIN_LENGTH:
        problems.append("shorter than " + str(MIN_LENGTH) + " characters")
    if len(pwd) > MAX_LENGTH:
        problems.append("longer than " + str(MAX_LENGTH) + " characters")

    has_upper = False
    has_lower = False
    has_digit = False
    has_special = False
    has_space = False
    for ch in pwd:
        if ch.isupper():
            has_upper = True
        elif ch.islower():
            has_lower = True
        elif ch.isdigit():
            has_digit = True
        elif ch in SPECIALS:
            has_special = True
        elif ch == " ":
            has_space = True

    if not has_upper:
        problems.append("no uppercase letter")
    if not has_lower:
        problems.append("no lowercase letter")
    if not has_digit:
        problems.append("no digit")
    if not has_special:
        problems.append("no special character (" + SPECIALS + ")")
    if has_space:
        problems.append("contains a space")

    for i in range(len(pwd) - 2):
        if pwd[i] == pwd[i + 1] and pwd[i + 1] == pwd[i + 2]:
            problems.append("character '" + pwd[i] + "' repeated 3+ times in a row")
            break

    lowered = pwd.lower()
    for word in PROHIBITED:
        if word in lowered:
            problems.append("contains prohibited text '" + word + "'")

    if len(problems) == 0:
        print("'" + pwd + "' -> VALID")
        valid_count = valid_count + 1
    else:
        print("'" + pwd + "' -> INVALID")
        for p in problems:
            print("     - " + p)

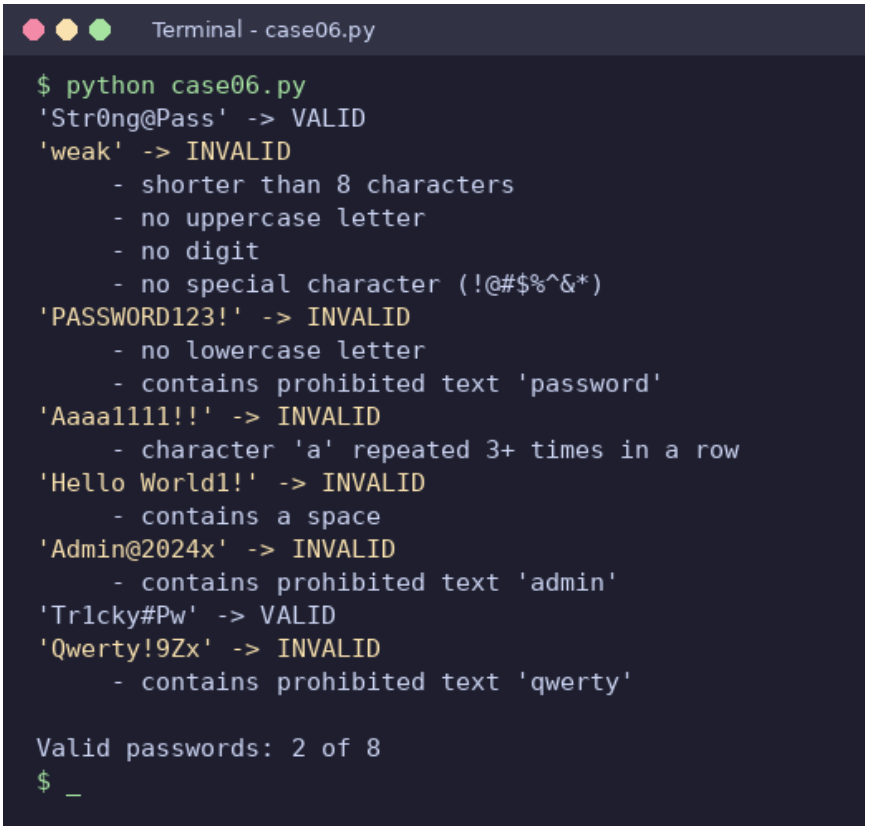
print()
print("Valid passwords:", valid_count, "of", len(passwords))
```

8. Test Cases

#	Test case	Input / data	Expected result	Observed output (actual)	Status
1	Default sample list	8 passwords	2 of 8 valid	`'Str0ng@Pass' -> VALID | Valid passwords: 2 of 8`	**PASS**
2	Minimal valid password	Abcdef1!	Exactly 8 characters, all categories	`'Abcdef1!' -> VALID`	**PASS**
3	Missing special character	Abcdef1	Two failures reported	`- no special character (!@#$%^&*) | - shorter than 8 characters`	**PASS**
4	Empty password	(empty string)	All categories missing	`- shorter than 8 characters | - no uppercase letter | - no digit`	**PASS**
5	Too long	24 characters	Length maximum enforced	`- longer than 20 characters`	**PASS**
6	Repeated characters	Aaaa1111!!	Repeat detected	`- character 'a' repeated 3+ times in a row`	**PASS**
7	Prohibited text, any case	PASSWORD123!	Case-insensitive match	`- contains prohibited text 'password' | - no lowercase letter`	**PASS**
8	Space inside	Hello World1!	Spaces not allowed	`- contains a space`	**PASS**

9. Output Screenshot

Terminal output of `python case06.py` using the default data:



10. Conclusion

The engine reports every violated rule for each password rather than only the first, which is exactly what a user needs to fix a weak password. It exercises string traversal, indexing, slicing-style comparisons, membership tests and list handling.